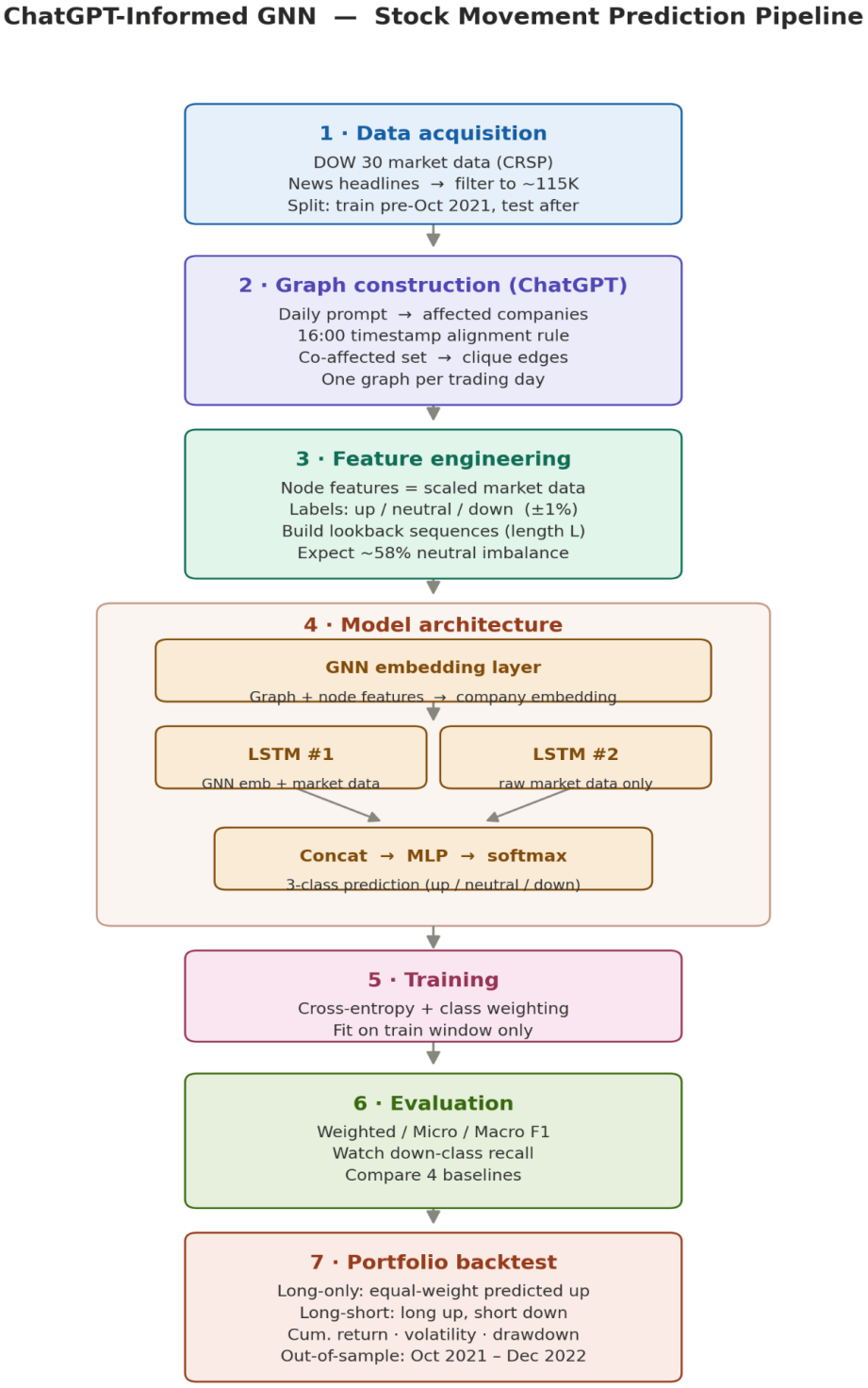
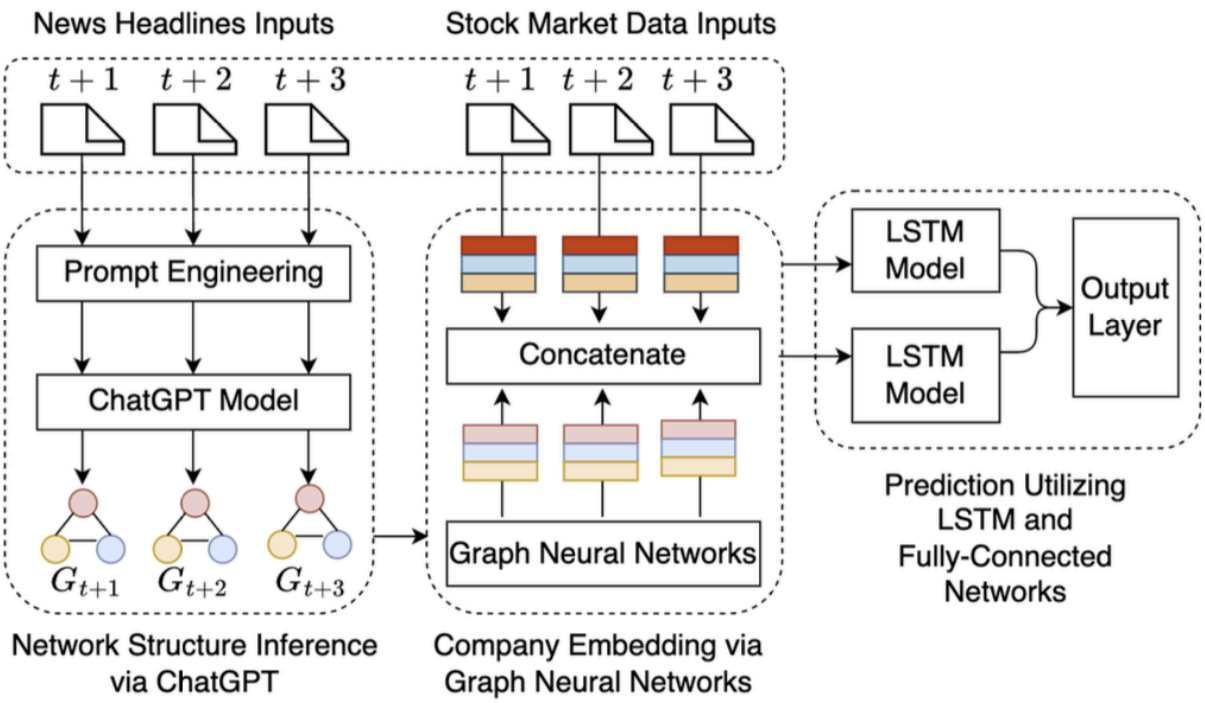
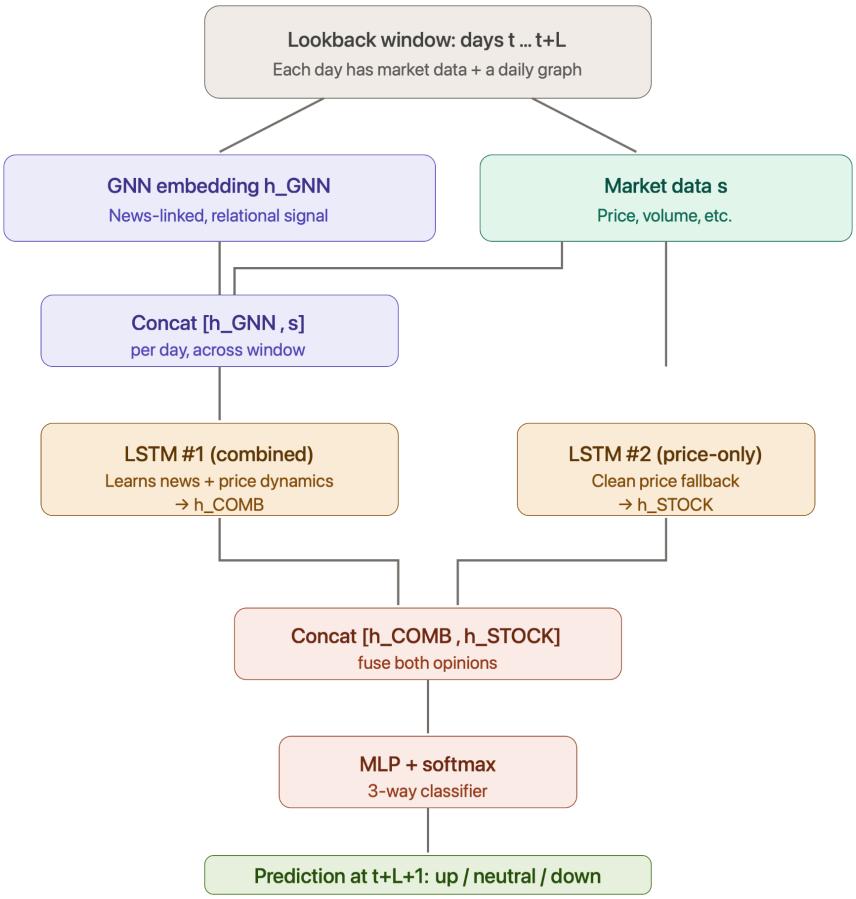
Paper 1 — ChatGPT-Informed GNN for Stock Movement Prediction (Chen et al., 2023)




News Headlines Inputs
Stock Market Data Inputs
$t+1$ $t+2$ $t+3$
$t+1$ $t+2$ $t+3$
Prompt Engineering
ChatGPT Model
G_{t+1} G_{t+2} G_{t+3}
Concatenate
Graph Neural Networks
LSTM Model
LSTM Model
Output Layer
Prediction Utilizing LSTM and Fully-Connected Networks
Network Structure Inference via ChatGPT
Company Embedding via Graph Neural Networks


Lookback window: days t ... t+L
Each day has market data + a daily graph
GNN embedding h_GNN
News-linked, relational signal
Market data s
Price, volume, etc.
Concat [h_GNN , s]
per day, across window
LSTM #1 (combined)
Learns news + price dynamics
→ h_COMB
LSTM #2 (price-only)
Clean price fallback
→ h_STOCK
Concat [h_COMB , h_STOCK]
fuse both opinions
MLP + softmax
3-way classifier
Prediction at t+L+1: up / neutral / down

1. Stock market data (numerical) — daily per-company values for the DOW 30: open, close, high, low, daily ask and bid, volume, and ordinary dividend amount. This is the node features for the GNN and the input to both LSTMs.

2. Financial news headlines (text) — daily headlines with a precise timestamp each, filtered to those mentioning a DOW 30 company. This is what you feed ChatGPT to build the daily graphs. (main Obstacle tog et this data we need to check RavenPack, Bloomberg, LSEG/Refinitiv (Reuters News),

3. Labels (you compute these) — no source needed; you derive up/neutral/down from the close-to-close return using the ±1% thresholds. Free.

ChatGPT-Informed GNN — S&P 500 Execution Plan (Point-in-Time)

0. Scope. Targets: S&P 500 using point-in-time membership (actual members on each date, not today's list). Multi-year window (e.g. 2018–2023), time-ordered split, test starts at/after the LLM's knowledge cutoff. Task: 3-class next-day movement (up/neutral/down), thresholds ±1%. Success = leak-free F1 metrics + a transaction-cost-aware backtest.

1. Point-in-time membership (do this first). Reconstruct the exact member list for every date, including companies that were later removed or delisted. Sources: CRSP/Compustat via WRDS (institutional), Norgate/Sharadar (~retail), or community add/remove datasets (free, verify against a second source). *Trap:* building the universe from today's 500 names silently deletes every failure and inflates returns — point-in-time membership with delisted tickers is what keeps the result honest.

2. Data. (a) Daily OHLCV + dividends for every name on the days it was a member, including delisted tickers (free per-ticker APIs often drop these). (b) Timestamped news headlines, filtered to companies that were members *on that date*(membership-aware join, not a static ticker set). (c) Labels derived from close-to-close return; use the delisting return on exit days so failures aren't dropped.

3. News→graph (ChatGPT). Filter headlines to current members → apply the 16:00 alignment rule (pre-16:00 predicts t+1, post-16:00 predicts t+2) → prompt the LLM to return affected companies as JSON → build a daily graph (nodes = members, edge between any two co-affected names). Expect a sparse graph; handle isolated nodes explicitly. Pin one model

version, temperature 0, cache every response, freeze as a versioned artifact. Cost is now a real budget line — estimate tokens × days, use the Batch API (50% off).

4. Features. Variable node set per day (membership shifts). Scale on train only. Lookback L (start 5–10 days); predict t+L+1 from t…t+L. Align graph/features/labels on the same trading-day index.

5. Model. GNN (2–3 layers viable at 500 nodes) → embedding h_GNN. Branch A: concat [h_GNN, market data] → LSTM #1. Branch B: raw market data → LSTM #2 (clean fallback). Fuse [h_COMB, h_STOCK] → MLP → softmax. Start shallow; mini-batch over time.

6. Training. Cross-entropy with class weights (neutral dominates, or the model collapses to all-neutral). Joint backprop through GNN + both LSTMs + MLP. Time-ordered validation split, never random. Fix seeds, log configs, checkpoint.

7. Evaluation. Weighted/Micro/Macro F1 + full confusion matrix. Watch down-class recall — high Micro F1 can just mean "predicts neutral." Build baselines (raw ChatGPT sentiment, news-embedding MLP, stock-only LSTM, ARIMA). Slice by sector/size/liquidity.

8. Backtest. Long-only (equal-weight predicted-up) and long-short (return, volatility, Sharpe, max drawdown). Include transaction costs — 500 names rebalanced daily churns heavily. Apply delisting returns on exit; note short-borrow assumptions.

9. Leakage audit. Survivorship bias (#1 risk), membership look-ahead, the 16:00 rule, train-only scaling, LLM cutoff vs test window, label boundary.

Watch-points before going live (don't skip when the time comes):

- **Data latency & point-in-time correctness:** live needs streaming news and real-time membership changes with no restated/backfilled values; free backfill sources won't deliver intraday.
- **LLM on a deadline:** graph-building moves from cheap offline batch to synchronous calls before open/close — budget for latency and an API-outage fallback.
- **Transaction costs, slippage, spread, market impact:** model them seriously; daily 500-name rebalancing is high-turnover and quietly erodes the backtest edge.
- **Capacity & short borrow:** can you actually fill the trades, and can you borrow to short smaller names?
- **Model decay / regime shift:** a model trained on one period degrades — plan scheduled retraining and live performance monitoring.
- **Risk controls & infrastructure:** position limits, stop-losses, drawdown circuit breakers, exposure/sector caps, broker API, reconciliation, monitoring, failover.
- **Paper-trade first:** run the full live pipeline against the market with no real capital for a meaningful stretch before committing money.

Cost:

Point-in-time membership + delisted tickers: free if you have WRDS (CRSP); otherwise Norgate/Sharadar are roughly $30–150/month, or community datasets free (verify against a second source).
Market data (OHLCV incl. delisted): same sources; free via WRDS, low-cost via Sharadar/Norgate, or free-with-caveats via yfinance.
News headlines at 500-name scale: free (GDELT) up to mid-tier paid ($30–100/month for Polygon/Tiingo/Marketaux) to enterprise.
LLM graph-building (OpenAI API): the one cost unique to this paper. At 500 names you pull far more news, so more tokens per day. Using a budget model (~$0.10–0.15 per 1M input tokens) and the Batch API (50% off), a multi-year run is still modest — plausibly low tens to low hundreds of dollars depending on how many headlines you feed per day. Estimate tokens × days before committing.

Computation : google colab pro+ or google VM []

https://github.com/ZihanChen1995/ChatGPT-GNN-StockPredict/tree/main

The issue is the news data. We need multiple place to collect it. In the paper they didn't mention how the writer got the data. Alpha vantage, WRDS and Bloomberg all of them have news headlines which can be used.